

ML/AI Interview Handbook

Quick-Reference Notes — Week 1 & Week 2

Prepared for: Gireesh Adireddi

Format: Definition • When to Use • Key Parameters • Simple Explanation • Interview
One-Liner

Contents

WEEK 1 — Data Foundations

1. NumPy
2. Pandas
3. Matplotlib
4. Seaborn
5. EDA

WEEK 2 — Classical ML Algorithms & Feature Engineering

6. Linear Regression
7. Logistic Regression
8. Decision Trees
9. Random Forest
10. Feature Engineering
11. Pipelines
12. Cross-Validation
13. GridSearchCV
14. Gradient Boosting
15. XGBoost
16. StandardScaler
17. PCA
18. t-SNE
19. K-Means Clustering
20. Baseline Models

WEEK 1 — Data Foundations

1. NumPy

Definition

A Python library for fast numerical computing, built around the **array** (ndarray) instead of Python lists.

When to Use

Whenever you need fast math operations on large sets of numbers — matrix operations, statistics, or as the backbone under Pandas/Scikit-learn/TensorFlow.

Key Parameters

- **dtype** — data type of array elements (int32, float64, etc.)
- **shape** — dimensions of the array, e.g. (rows, columns)
- **axis** — direction of operation — axis=0 is column-wise, axis=1 is row-wise

Simple Explanation (Analogy)

Think of a Python list as a single queue of people, and a NumPy array as an entire stadium seating chart — organized in rows and columns, so you can instantly grab 'all of row 3' or 'all of column 2' instead of walking through every seat one by one.

Interview One-Liner

NumPy gives vectorized (batch) math operations on arrays, which is why it's dramatically faster than looping through plain Python lists.

2. Pandas

Definition

A library for working with structured, tabular data using **DataFrames** (rows & columns, like Excel) and **Series** (a single column).

When to Use

Any time you're loading, cleaning, filtering, grouping, or transforming tabular/CSV-style data before feeding it to a model.

Key Parameters

- **index** — the row labels of a DataFrame
- **columns** — the column labels
- **axis** — 0 = operate down rows, 1 = operate across columns (same idea as NumPy)

Simple Explanation (Analogy)

Pandas is like an Excel sheet you can control with code — instead of manually clicking filters, you write one line like `df[df['price'] > 100]` and it filters instantly, even on millions of rows.

Interview One-Liner

Pandas = Excel for code — DataFrame is the full sheet, Series is a single column, and most work is filtering, grouping (groupby), and cleaning (fillna, dropna).

3. Matplotlib

Definition

The foundational Python library for creating static plots and charts (line, bar, scatter, histogram, etc.).

When to Use

Whenever you need full manual control over a plot's look — axis labels, custom colors, multiple subplots.

Key Parameters

- **figsize** — size of the plot (width, height) in inches
- **kind/plot type** — e.g. `plt.plot()`, `plt.bar()`, `plt.scatter()`, `plt.hist()`
- **xlabel/ylabel/title** — text labels for axes and chart title

Simple Explanation (Analogy)

Matplotlib is like a blank canvas and a full box of paints — powerful, but you decide every brushstroke yourself (every label, every color) rather than getting a ready-made picture.

Interview One-Liner

Matplotlib is the base plotting engine in Python — most other plotting libraries (like Seaborn) are actually built on top of it.

4. Seaborn

Definition

A statistical visualization library built on top of Matplotlib, designed to make attractive, informative plots with much less code.

When to Use

When you want quick, good-looking statistical plots — correlation heatmaps, distribution plots, boxplots — without manually styling everything.

Key Parameters

- **hue** — adds a third categorical variable as color grouping
- **data** — the DataFrame the plot pulls from
- **kind** — type of plot for functions like `sns.catplot()`, e.g. 'box', 'violin', 'bar'

Simple Explanation (Analogy)

If Matplotlib is a blank canvas and paints, Seaborn is like a paint-by-numbers kit tailored for data — you get a beautiful, statistically meaningful picture with far fewer manual steps.

Interview One-Liner

Seaborn = Matplotlib + built-in statistical smarts + nicer default styling, especially good for heatmaps and distribution plots.

5. EDA (Exploratory Data Analysis)

Definition

The process of investigating a dataset first — checking its shape, missing values, distributions, and relationships — before doing any modeling.

When to Use

Always, at the very start of every ML project — never skip straight to modeling on data you haven't looked at.

Simple Explanation (Analogy)

EDA is like a doctor doing a full check-up before prescribing any medicine — you can't fix problems (missing values, outliers, wrong data types) you haven't diagnosed yet.

Interview One-Liner

EDA = understanding your data's shape, quality, and patterns (nulls, outliers, distributions, correlations) before building any model — 'garbage in, garbage out' is exactly why this step matters.

WEEK 2 — Classical ML Algorithms & Feature Engineering

6. Linear Regression

Definition

A supervised ML algorithm that predicts a **continuous numeric value** by fitting a straight line (or hyperplane) through the data.

When to Use

Predicting numbers — house prices, sales figures, temperatures — when the relationship between input and output is roughly linear.

Key Parameters

- **fit_intercept** — whether to calculate the intercept (bias) term — usually True
- **coef_** — the learned weight/slope for each feature (after training)
- **intercept_** — the learned bias/starting point of the line

Simple Explanation (Analogy)

Imagine plotting runs scored vs. balls faced for a batsman and drawing the single straight line that best represents the trend — that line lets you estimate runs for any new ball count.

Interview One-Liner

Linear Regression fits $y = mx + c$ (or a multi-feature version of it) to minimize the squared error between predicted and actual values — used for continuous-value prediction, evaluated with RMSE/MAE/R².

7. Logistic Regression

Definition

Despite the name, a **classification** algorithm (not regression) — predicts the probability of a data point belonging to a class (e.g., yes/no, spam/not spam) using the sigmoid function.

When to Use

Binary or multi-class classification problems, especially when you want a fast, interpretable baseline model.

Key Parameters

- **C** — inverse of regularization strength — smaller C = stronger regularization
- **penalty** — type of regularization — 'l1', 'l2', or 'elasticnet'
- **solver** — optimization algorithm used, e.g. 'lbfgs', 'liblinear'
- **max_iter** — maximum number of iterations for the solver to converge

Simple Explanation (Analogy)

Instead of predicting an exact number, Logistic Regression is like an umpire giving a percentage confidence — '85% sure this is OUT' — then you set a threshold (usually 50%) to make the final yes/no call.

Interview One-Liner

Logistic Regression uses the sigmoid function to squash any input into a probability between 0 and 1, then applies a threshold (commonly 0.5) to classify — it's linear at its core, just wrapped for classification.

8. Decision Trees

Definition

A model that splits data into branches based on feature conditions (like a flowchart of if-else questions) to reach a final prediction at the 'leaf'.

When to Use

When you want an easily interpretable model, or as a building block for ensemble methods (Random Forest, XGBoost).

Key Parameters

- **max_depth** — maximum depth of the tree — controls overfitting
- **criterion** — how splits are evaluated — 'gini' or 'entropy' for classification, 'mse' for regression
- **min_samples_split** — minimum samples required to split a node
- **min_samples_leaf** — minimum samples required at a leaf node

Simple Explanation (Analogy)

It's like the '20 questions' game — 'Is it bigger than a car? No. Does it fly? Yes.' — each answer narrows down the possibilities until you land on the final answer (the leaf).

Interview One-Liner

Decision Trees split data recursively on the feature/threshold that best separates classes (using Gini impurity or entropy) — simple and interpretable, but prone to overfitting if too deep.

9. Random Forest

Definition

An **ensemble** of many Decision Trees, where each tree is trained on a random subset of data/features, and the final prediction is the average (regression) or majority vote (classification) of all trees.

When to Use

When a single Decision Tree overfits — Random Forest reduces variance and generally gives better, more stable accuracy.

Key Parameters

- **n_estimators** — number of trees in the forest
- **max_depth** — maximum depth of each individual tree
- **max_features** — number of features considered at each split
- **random_state** — seed for reproducibility

Simple Explanation (Analogy)

Instead of asking ONE friend's opinion on a movie (one Decision Tree, which might be biased), you ask 100 friends and go with the majority vote — much more reliable than trusting a single opinion.

Interview One-Liner

Random Forest = many Decision Trees trained on random subsets (bagging), combined by voting/averaging — reduces overfitting compared to a single tree, at the cost of interpretability and speed.

10. Feature Engineering

Definition

The process of creating, transforming, or selecting input features to improve model performance — e.g., combining columns, extracting date parts, encoding categories.

When to Use

Always, before modeling — good feature engineering often improves accuracy more than switching algorithms.

Simple Explanation (Analogy)

If raw data is unprepared ingredients, feature engineering is the chopping, marinating, and seasoning before cooking — the same base ingredients (data) can make a mediocre or an excellent dish (model) depending on this prep work.

Interview One-Liner

Feature Engineering = transforming raw data into more useful inputs (e.g., extracting 'day of week' from a date, one-hot encoding categories, creating ratio features) — often the biggest lever for model performance.

11. Pipelines

Definition

A Scikit-learn tool that chains multiple preprocessing + modeling steps into a single object, so data flows through all steps in the correct order automatically.

When to Use

Whenever your workflow has multiple steps (scaling, encoding, then modeling) — pipelines prevent data leakage and keep code clean and reusable.

Key Parameters

- **steps** — list of (name, transformer/estimator) tuples executed in order
- **fit()/predict()** — calling these on the pipeline runs ALL steps automatically, in order

Simple Explanation (Analogy)

A pipeline is like an assembly line in a factory — raw material (data) goes in one end, passes through each station (scaling, encoding, model) automatically, and a finished product (prediction) comes out the other end — no manual handling in between.

Interview One-Liner

Pipeline chains preprocessing and modeling steps together so fit/predict/transform happen consistently and in the correct order — critical for avoiding data leakage during cross-validation.

12. Cross-Validation

Definition

A technique to evaluate model performance by splitting data into multiple folds, training on some and testing on the rest, rotating through all folds.

When to Use

Whenever you want a more reliable estimate of model performance than a single train/test split — especially with limited data.

Key Parameters

- **cv** — number of folds (e.g., cv=5 means 5-fold cross-validation)
- **scoring** — metric used to evaluate each fold, e.g. 'accuracy', 'r2', 'neg_mean_squared_error'

Simple Explanation (Analogy)

Instead of judging a batsman's form from just ONE innings (a single train/test split), you look at his last 5 innings (5 folds) and average the performance — a much fairer judgment of true skill.

Interview One-Liner

Cross-validation (commonly k-fold) splits data into k parts, trains on k-1 and tests on the remaining part, rotating through all k combinations — gives a more robust performance estimate than one split.

13. GridSearchCV

Definition

A method that automatically tries every combination of specified hyperparameters (combined with cross-validation) to find the best-performing combination.

When to Use

When you need to tune hyperparameters (like `max_depth`, `n_estimators`) systematically instead of guessing manually.

Key Parameters

- **param_grid** — dictionary of hyperparameters and the values to try for each
- **cv** — number of cross-validation folds used to evaluate each combination
- **scoring** — metric used to decide which combination is 'best'
- **best_params_** — the winning hyperparameter combination after search

Simple Explanation (Analogy)

Like trying every possible batting order combination in practice matches to find the one that scores the most runs on average — GridSearchCV brute-forces through every hyperparameter combo to find the best one.

Interview One-Liner

GridSearchCV exhaustively tests every combination in param_grid using cross-validation, then reports the best_params_ — thorough but can be slow with many parameters/values.

14. Gradient Boosting

Definition

An ensemble technique that builds trees **sequentially**, where each new tree corrects the errors (residuals) made by the previous trees.

When to Use

When you need high accuracy and are willing to trade off some training speed and interpretability — very strong on structured/tabular data.

Key Parameters

- **n_estimators** — number of boosting stages (trees) to build
- **learning_rate** — shrinks the contribution of each tree — smaller values need more trees but generalize better
- **max_depth** — depth of each individual tree (usually kept shallow, e.g. 3-5)

Simple Explanation (Analogy)

Unlike Random Forest (many independent opinions averaged), Gradient Boosting is like a student redoing a test — first attempt gets some answers wrong, second attempt specifically focuses on fixing those wrong answers, and so on, each round improving on the last.

Interview One-Liner

Gradient Boosting builds trees sequentially, each one fitting the residual errors of the previous ensemble, controlled by `learning_rate` — a foundational idea behind XGBoost/LightGBM/CatBoost.

15. XGBoost

Definition

'Extreme Gradient Boosting' — an optimized, regularized, and much faster implementation of Gradient Boosting, widely used for tabular data competitions and production models.

When to Use

Structured/tabular data problems where you want top-tier accuracy with reasonable training speed — a go-to model for Kaggle-style competitions.

Key Parameters

- **n_estimators** — number of boosting rounds (trees)
- **learning_rate (eta)** — step size shrinkage to prevent overfitting
- **max_depth** — maximum depth of each tree
- **subsample** — fraction of training data used per tree (prevents overfitting)
- **colsample_bytree** — fraction of features used per tree

Simple Explanation (Analogy)

If Gradient Boosting is a student improving test-by-test, XGBoost is that same student but with a much better study system — built-in regularization (to avoid over-studying/overfitting one topic) and parallel processing (studying multiple subjects at once) making it faster and more accurate.

Interview One-Liner

XGBoost is Gradient Boosting with added regularization (L1/L2), handling of missing values, and parallelized tree construction — usually the strongest 'out of the box' choice for tabular ML problems.

16. StandardScaler

Definition

A preprocessing tool that transforms features to have **mean = 0** and **standard deviation = 1** (i.e., standardization/z-score scaling).

When to Use

Before feeding data into distance-based or gradient-based algorithms (SVM, KNN, Logistic Regression, Neural Networks, PCA) that are sensitive to feature scale.

Key Parameters

- **with_mean** — whether to center the data by subtracting the mean (default True)
- **with_std** — whether to scale to unit variance (default True)

Simple Explanation (Analogy)

If one feature is 'age' (0-100) and another is 'salary' (0-1,000,000), a model might wrongly think salary matters way more just because its numbers are bigger — StandardScaler puts every feature on the same measuring scale, like converting everyone's runs and wickets into a common 'performance rating' before comparing.

Interview One-Liner

StandardScaler transforms each feature as $(x - \text{mean}) / \text{std_dev}$, so all features contribute on a comparable scale — essential for distance-based and gradient-based algorithms.

17. PCA (Principal Component Analysis)

Definition

A **dimensionality reduction** technique that transforms many correlated features into fewer new features (principal components) that capture most of the original variance/information.

When to Use

When you have too many features (curse of dimensionality), want to speed up training, reduce noise, or visualize high-dimensional data in 2D/3D.

Key Parameters

- **n_components** — number of principal components to keep (or a variance % like 0.95 to auto-select)
- **explained_variance_ratio_** — how much information/variance each component captures

Simple Explanation (Analogy)

Imagine summarizing a 50-page match report into a 1-page highlights summary that still captures the key moments — PCA does that for data, compressing many columns into a few 'super-columns' that retain most of the original information.

Interview One-Liner

PCA finds new axes (principal components) that capture maximum variance in the data, letting you reduce dimensions while retaining most of the information — commonly used before modeling or for visualization.

18. t-SNE (t-Distributed Stochastic Neighbor Embedding)

Definition

A dimensionality reduction technique mainly used for **visualization**, which preserves local relationships (similar points stay close together) when projecting high-dimensional data into 2D/3D.

When to Use

When you want to visually explore clusters/patterns in high-dimensional data (e.g., visualizing image embeddings or word vectors) — NOT typically used as a preprocessing step before modeling like PCA.

Key Parameters

- **n_components** — number of dimensions to reduce to (usually 2 for visualization)
- **perplexity** — roughly balances attention between local and global structure (typical range 5-50)
- **learning_rate** — controls how much the embedding positions adjust each step

Simple Explanation (Analogy)

If PCA is like a wide-angle photo capturing the overall scene, t-SNE is like a photo that makes sure people who were standing close together in real life still appear close together in the photo — great for seeing 'natural groupings,' even if it distorts overall distances.

Interview One-Liner

t-SNE is primarily a visualization tool (not for feeding into further models) that preserves local neighborhood structure — useful for spotting clusters in high-dimensional embeddings.

19. K-Means Clustering

Definition

An **unsupervised** algorithm that groups data into K clusters by repeatedly assigning points to the nearest cluster center (centroid) and updating centroids.

When to Use

When you want to discover natural groupings in unlabeled data — customer segmentation, grouping similar products, etc.

Key Parameters

- **n_clusters** — the number of clusters (K) to form — must be chosen beforehand (e.g., via Elbow Method)
- **init** — method for initializing centroids, e.g. 'k-means++' (smarter default)
- **n_init** — number of times the algorithm runs with different centroid seeds, keeping the best result
- **max_iter** — maximum iterations per run

Simple Explanation (Analogy)

Imagine dropping K captains randomly on a cricket ground full of players, then repeatedly reassigning each player to their nearest captain and moving each captain to the center of their new team — after a few rounds, you naturally end up with well-separated, sensible teams.

Interview One-Liner

K-Means alternates between assigning points to the nearest centroid and recomputing centroids until convergence — unsupervised, requires choosing K upfront (often via the Elbow Method), sensitive to initialization and outliers.

20. Baseline Models

Definition

A simple, minimal-effort model (e.g., predicting the mean/most frequent class, or a plain Linear/Logistic Regression) used as a reference point before building complex models.

When to Use

Always build one FIRST, before any complex model — if your fancy model can't beat the baseline, something is wrong.

Simple Explanation (Analogy)

Before claiming you're a great batsman, you first check what an average club player would score in the same conditions — the baseline. If your 'advanced technique' doesn't outscore that average player, it's not actually adding value.

Interview One-Liner

A Baseline Model sets the minimum bar for performance (e.g., predicting the mean, or a simple model) — every complex model must be justified by beating this baseline, otherwise the added complexity isn't worth it.
